2303A510A1 - Bhanu
Assignment - 11.1
Batch - 14

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task 1 - Stack Implementation

Prompt:

- Generate a Python class named Stack implementing a LIFO data structure.
- Requirements: Use a Python list internally.
- Methods: push(item), pop(), peek(), is_empty(). Handle empty stack properly. Add proper docstrings and comments. Include example usage. Follow PEP8 standards.

Code:

```
class Stack:
    """
    A class representing a Stack data structure (LIFO - Last In, First
Out).
    Internally uses a Python list to store the elements of the stack.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self._stack = []
    def push(self, item):
        """
        Add an item to the top of the stack.
        Parameters:
        item: The item to be added to the stack.
```

```
        """
        self._stack.append(item)
    def pop(self):
        """
        Remove and return the item at the top of the stack.
        Returns:
        The item at the top of the stack.
        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self._stack.pop()
    def peek(self):
        """
        Return the item at the top of the stack without removing it.
        Returns:
        The item at the top of the stack.
        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self._stack[-1]
    def is_empty(self):
        """
        Check if the stack is empty.
        Returns:
        True if the stack is empty, False otherwise.
        """
        return len(self._stack) == 0
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    stack.push(4)
    stack.push(5)
    stack.push(6)
    print(stack.peek())
```

```
    print(stack.pop())
    print(stack.is_empty())
    print(stack.pop())
    print(stack.is_empty())
    print(stack._stack)
```

Output:

```
6
6
False
5
False
[1, 2, 3, 4]
```

Task 2 – Queue Implementation

Prompt:

- Generate a Python class named Queue implementing FIFO behavior using a list.
- Requirements: enqueue(item), dequeue(), peek(), size(). Handle empty queue cases safely. Add class and method docstrings. Add time complexity comments. Include example usage. Follow PEP8 standards.

Code:

```
class Queue:
    """A class representing a Queue data structure implementing FIFO
behavior using a list."""
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []
```

```
    def enqueue(self, item):
        """Add an item to the end of the queue.
        Time Complexity: O(1)
        Args:
            item: The item to be added to the queue.
        """
        self.items.append(item)
    def dequeue(self):
        """Remove and return the item at the front of the queue.
        If the queue is empty, return None.
        Time Complexity: O(n) due to list shifting
        Returns:
            The item at the front of the queue or None if the queue is
empty.
        """
        if self.is_empty():
            return None
        return self.items.pop(0)
    def peek(self):
        """Return the item at the front of the queue without removing it.
        If the queue is empty, return None.
        Time Complexity: O(1)
        Returns:
            The item at the front of the queue or None if the queue is
empty.
        """
        if self.is_empty():
            return None
        return self.items[0]
    def size(self):
        """Return the number of items in the queue.
        Time Complexity: O(1)
        Returns:
            The number of items in the queue.
        """
        return len(self.items)
    def is_empty(self):
        """Check if the queue is empty.
        Time Complexity: O(1)
        Returns:
```

```
            True if the queue is empty, False otherwise.
        """
        return len(self.items) == 0
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)
    queue.enqueue(4)
    queue.enqueue(5)
    queue.enqueue(6)
    print(queue.peek())
    print(queue.dequeue())
    print(queue.size())
    print(queue.is_empty())
    print(queue.dequeue())
    print(queue.items)
```

Output:

```
1
1
5
False
2
[3, 4, 5, 6]
```

Task 3 – Linked List

Prompt:

- Generate Python code for a Singly Linked List.

- Requirements: Create a Node class with attributes data and next. Create a LinkedList class. Implement insert(data) to insert at the end. Implement display() to print the list.Add proper comments and docstrings. Provide sample usage. Follow PEP8 standards.

Code:

```
class Node:
    """A class representing a node in a singly linked list.
    Attributes:
        data: The data stored in the node.
        next: A reference to the next node in the list.
    """
    def __init__(self, data):
        """Initialize a new node with the given data.
        Args:
            data: The data to be stored in the node.
        """
        self.data = data
        self.next = None
class LinkedList:
    """A class representing a singly linked list.
    Attributes:
        head: The head node of the linked list.
    """
    def __init__(self):
        """Initialize an empty linked list."""
        self.head = None
    def insert(self, data):
        """Insert a new node with the given data at the end of the list.
        Args:
            data: The data to be stored in the new node.
        """
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
```

```
            last_node = last_node.next
        last_node.next = new_node
    def display(self):
        """Print the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List contents:")
    linked_list.display()
```

Output:

```
Linked List contents:
10 20 30
```

Task 4 – Binary Search Tree (BST)

Prompt:

- Create a Python class for a Binary Search Tree.
- Requirements: Implement recursive insert(value). Implement an inorder traversal method. Maintain BST property. Add proper docstrings. Include example usage inserting multiple values. Follow PEP8 standards.

Code:

```
class TreeNode:
    """A node in the binary search tree."""
    def __init__(self, value):
        """Initialize a TreeNode with a given value."""
        self.value = value
        self.left = None
        self.right = None
class BinarySearchTree:
    """A binary search tree implementation."""
    def __init__(self):
        """Initialize an empty binary search tree."""
        self.root = None
    def insert(self, value):
        """Insert a value into the binary search tree."""
        if self.root is None:
            self.root = TreeNode(value)
        else:
            self._insert_recursive(self.root, value)
    def _insert_recursive(self, node, value):
        """Helper method to insert a value recursively."""
        if value < node.value:
            if node.left is None:
                node.left = TreeNode(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = TreeNode(value)
            else:
                self._insert_recursive(node.right, value)
    def inorder_traversal(self):
        """Perform an inorder traversal of the tree and return a list of
values."""
        return self._inorder_recursive(self.root)
    def _inorder_recursive(self, node):
        """Helper method to perform an inorder traversal recursively."""
        values = []
        if node is not None:
            values.extend(self._inorder_recursive(node.left))
            values.append(node.value)
```

```
            values.extend(self._inorder_recursive(node.right))
        return values
if __name__ == "__main__":
    bst = BinarySearchTree()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("Inorder Traversal:", bst.inorder_traversal())
```

Output:

```
Inorder Traversal: [2, 3, 4, 5, 6, 7, 8]
```

Task 5 – Hash Table

Prompt:

- Generate a Python HashTable class.
- Requirements: Use list of lists for collision handling (chaining). Implement insert(key, value), search(key), delete(key). Use modulo-based hash function. Add comments explaining collision handling. Include example usage. Follow PEP8 standards.

Code:

```
class HashTable:
    def __init__(self, size=10):
        """Initialize the hash table with a specified size."""
        self.size = size
        self.table = [[] for _ in range(size)]
    def hash_function(self, key):
        """Generate a hash value for the given key using modulo-based
hashing."""
```

```
        return hash(key) % self.size
    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        self.table[index].append((key, value))
    def search(self, key):
        """Search for a value by its key in the hash table."""
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None
    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    print(hash_table.search("name"))
    print(hash_table.search("age"))
    print(hash_table.delete("name"))
    print(hash_table.search("name"))
```

Output:

```
Alice
30
True
None
```

Task 6 – Graph Representation

Prompt:

- Generate a Python Graph class using adjacency list representation.
- Requirements: Use dictionary for adjacency list.
- Methods: add_vertex(v), add_edge(v1, v2), display(). Undirected graph. Add docstrings and comments. Include example usage. Follow PEP8 standards.

Code:

```
class Graph:
    """A class to represent an undirected graph using an adjacency
list."""
    def __init__(self):
        """Initialize the graph with an empty adjacency list."""
        self.adjacency_list = {}
    def add_vertex(self, v):
        """Add a vertex to the graph.
        Args:
            v: The vertex to be added.
        """
        if v not in self.adjacency_list:
            self.adjacency_list[v] = []
    def add_edge(self, v1, v2):
        """Add an undirected edge between two vertices in the graph.
        Args:
            v1: The first vertex.
            v2: The second vertex.
        """
        if v1 not in self.adjacency_list:
            self.add_vertex(v1)
        if v2 not in self.adjacency_list:
            self.add_vertex(v2)
        if v2 not in self.adjacency_list[v1]:
            self.adjacency_list[v1].append(v2)
        if v1 not in self.adjacency_list[v2]:
```

```
            self.adjacency_list[v2].append(v1)
    def display(self):
        """Display the adjacency list of the graph."""
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex}: {edges}")
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "C")
    graph.display()
```

Output:

```
A: ['B', 'C']
B: ['A', 'C']
C: ['A', 'B']
```

Task 7 – Priority Queue

Prompt:

- Generate a Python PriorityQueue class using heapq module.
- Requirements: Use min-heap.
- Methods: enqueue(item, priority), dequeue(), display(). Add docstrings and comments. Include example usage. Explain why heap ensures efficient priority handling. Follow PEP8 standards.

Code:

```
import heapq
class PriorityQueue:
    """
    A Priority Queue implementation using a min-heap.
```

```
    This class allows you to enqueue items with a specified priority and
dequeue the item with the highest priority (lowest value).
    """
    def __init__(self):
        """Initialize an empty priority queue."""
        self._queue = []
        self._index = 0
    def enqueue(self, item, priority):
        """
        Add an item to the priority queue with a given priority.
        Args:
            item: The item to be added to the queue.
            priority: The priority of the item (lower values indicate
higher priority).
        """
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1
    def dequeue(self):
        """
        Remove and return the item with the highest priority from the
queue.
        Returns:
            The item with the highest priority.
        Raises:
            IndexError: If the queue is empty.
        """
        if not self._queue:
            raise IndexError("Dequeue from an empty priority queue")
        return heapq.heappop(self._queue)[2]
    def display(self):
        """Display all items in the priority queue."""
        print("Priority Queue:")
        for priority, index, item in sorted(self._queue):
            print(f"Priority: {priority}, Item: {item}")
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=3)
    print("Items in the priority queue:")
```

```
    pq.display()
    print("\nDequeueing items:")
    print(pq.dequeue())
    print(pq.dequeue())
    print(pq.dequeue())
```

Output:

```
Items in the priority queue:
Priority Queue:
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3

Dequeueing items:
Task 2
Task 1
Task 3
```

Task 8 – Deque

Prompt:

- Generate a Python class DequeDS using collections.deque.
- Requirements: insert_front(item), insert_rear(item), remove_front(), remove_rear(). Handle empty deque properly. Add docstrings and comments. Include example usage. Follow PEP8 standards.

Code:

```
from collections import deque
class DequeDS:
    """
    A class representing a double-ended queue (deque) data structure using
collections.deque.
    """
```

```python
    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()
    def insert_front(self, item):
        """
        Insert an item at the front of the deque.
        :param item: The item to be inserted.
        """
        self.deque.appendleft(item)
    def insert_rear(self, item):
        """
        Insert an item at the rear of the deque.
        :param item: The item to be inserted.
        """
        self.deque.append(item)
    def remove_front(self):
        """
        Remove and return the item at the front of the deque.
        :return: The item removed from the front of the deque.
        :raises IndexError: If the deque is empty.
        """
        if not self.deque:
            raise IndexError("Deque is empty. Cannot remove from front.")
        return self.deque.popleft()
    def remove_rear(self):
        """
        Remove and return the item at the rear of the deque.
        :return: The item removed from the rear of the deque.
        :raises IndexError: If the deque is empty.
        """
        if not self.deque:
            raise IndexError("Deque is empty. Cannot remove from rear.")
        return self.deque.pop()

if __name__ == "__main__":
    deque_ds = DequeDS()
    deque_ds.insert_rear(1)
    deque_ds.insert_rear(2)
    deque_ds.insert_front(0)
    print("Deque after insertions:", list(deque_ds.deque))
```

```
    print("Removed from front:", deque_ds.remove_front())
    print("Removed from rear:", deque_ds.remove_rear())
    print("Deque after removals:", list(deque_ds.deque))
```

Output:

```
Deque after insertions: [0, 1, 2]
Removed from front: 0
Removed from rear: 2
Deque after removals: [1]
```

Task 9: Real-Time Application Challenge – Choose the Right Data Structure

Prompt:

- Given these features:
    - Student Attendance Tracking
    - Event Registration System
    - Library Book Borrowing
    - Bus Scheduling System
    - Cafeteria Order Queue
- Select the most appropriate data structure from:
- Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque.
- Provide: A table mapping Feature → Data Structure → Justification. 2-3 sentence justification for each. Focus on real-world suitability and time complexity.

Code:

```
class CafeteriaOrderQueue:
    """A simple implementation of a cafeteria order queue system using a
list."""
```

```
    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = []
    def add_order(self, order):
        """Add an order to the queue.
        Args:
            order: The order to be added to the queue.
        """
        self.orders.append(order)
    def serve_order(self):
        """Serve the next order in the queue (FIFO).
        Returns:
            The next order if the queue is not empty; otherwise, raises an
exception.
        """
        if not self.orders:
            raise IndexError("No orders to serve")
        return self.orders.pop(0)
    def display_pending_orders(self):
        """Display all pending orders in the queue."""
        if not self.orders:
            print("No pending orders.")
        else:
            print("Pending Orders:")
            for order in self.orders:
                print(order)
if __name__ == "__main__":
    cafeteria_queue = CafeteriaOrderQueue()
    cafeteria_queue.add_order("Order 1: Coffee")
    cafeteria_queue.add_order("Order 2: Sandwich")
    cafeteria_queue.add_order("Order 3: Salad")
    cafeteria_queue.display_pending_orders()
    print("\nServing orders:")
    while True:
        try:
            print(cafeteria_queue.serve_order())
        except IndexError:
            print("All orders have been served.")
            break
    cafeteria_queue.display_pending_orders()
```

Output:

```
Pending Orders:
Order 1: Coffee
Order 2: Sandwich
Order 3: Salad

Serving orders:
Order 1: Coffee
Order 2: Sandwich
Order 3: Salad
All orders have been served.
No pending orders.
```

Justification:

Feature	Data Structure	Justification
Student Attendance Tracking	Deque	Students enter and exit from both ends efficiently.
Event Registration	Hash Table	Fast search and removal using student ID as key.
Library Book Borrowing	BST	Books sorted by ID/due date for ordered retrieval.
Bus Scheduling	Graph	Routes and stops form connected network.
Cafeteria Order Queue	Queue	Orders served in FIFO order.

Task 10: Smart E-Commerce Platform – Data Structure Challenge

Prompt:

- Generate a Python program implementing Product Search using Hash Table.
- Requirements: add_product(id, name), search_product(id), remove_product(id). Fast lookup using dictionary. Include comments and docstrings. Provide example usage.

Code:

```
class ProductSearchEngine:
    """A simple implementation of a product search engine using a hash
table."""
    def __init__(self):
        """Initialize an empty product search engine."""
        self.products = {}
    def add_product(self, product_id, product_name):
        """Add a product to the search engine.
        Args:
            product_id: The unique identifier for the product.
            product_name: The name of the product.
        """
        self.products[product_id] = product_name
    def search_product(self, product_id):
        """ Search for a product by its ID.
        Args:
            product_id: The unique identifier for the product to be
searched.
        Returns:
            The name of the product if found; otherwise, None.
        """
        return self.products.get(product_id)
    def delete_product(self, product_id):
        """Delete a product from the search engine by its ID.
```

```
        Args:
            product_id: The unique identifier for the product to be
deleted.
        """
        if product_id in self.products:
            del self.products[product_id]
    def display_products(self):
        """Display all products in the search engine."""
        if not self.products:
            print("No products available.")
        else:
            print("Products:")
            for product_id, product_name in self.products.items():
                print(f"ID: {product_id}, Name: {product_name}")
if __name__ == "__main__":
    search_engine = ProductSearchEngine()
    search_engine.add_product(1, "Laptop")
    search_engine.add_product(2, "Smartphone")
    search_engine.add_product(3, "Headphones")
    search_engine.display_products()
    print("\nSearching for product with ID 2:")
    print(search_engine.search_product(2))
    print("\nDeleting product with ID 1.")
    search_engine.delete_product(1)
    print("\nProducts after deletion:")
    search_engine.display_products()
```

Output:

```
Products:
ID: 1, Name: Laptop
ID: 2, Name: Smartphone
ID: 3, Name: Headphones

Searching for product with ID 2:
Smartphone

Deleting product with ID 1.

Products after deletion:
Products:
ID: 2, Name: Smartphone
ID: 3, Name: Headphones
```

Justification:

Feature	Data Structure	Justification
Shopping Cart	Linked List	Dynamic add/remove of products.
Order Processing	Queue	Orders processed FIFO.
Top-Selling Products	Priority Queue	Ranked by sales count.
Product Search	Hash Table	Fast lookup by product ID.
Delivery Route Planning	Graph	Warehouses and routes form network.